JobPortal

High-Level Design (HLD) & Low-Level Design (LLD)

Full-stack MERN job portal with AI match scoring

React 19 + Vite | Express + MongoDB | Groq LLaMA 3.3 | Cloudinary | Vercel / Render / Atlas

Generated from the repository: backend/ (routes, controllers, models) and frontend/src/ (components, hooks, redux).

1. What are HLD and LLD?

A design document has two levels. The **HLD** is the map: which parts exist and how they talk. The **LLD** is the blueprint of each part: exact schemas, endpoints, functions and rules, detailed enough to code from without asking questions.

	HLD (High-Level Design)	LLD (Low-Level Design)
Question it answers	What is the system made of? How do the pieces connect?	How is each piece built, exactly?
Audience	Anyone: new developer, reviewer, manager	Developers who will build or change the code
Contains	Architecture diagram, modules, main flows, tech choices and reasons, non-functional needs	Schemas, API tables, function names, validations, error codes, state shape, sequence diagrams
Mentions file names?	No	Yes - must match the repo one to one
Test	Still true if the backend were rewritten in Python	Names a field, file or function

Rules for writing the HLD

- **No code.** Boxes, arrows, tables and sentences. Naming a function means you have gone too deep.
- **Every external system gets a box**: here Vercel, Render, Atlas, Cloudinary, Groq and logo.dev.
- **Show 3-4 flows end to end**, the ones that define the product, not every flow.
- **Write the decisions and the reasons** (for example: JWT in an httpOnly cookie rather than localStorage).
- **One table of non-functional requirements**: auth, roles, limits, secrets, fallbacks.

Rules for writing the LLD

- **One chapter per HLD module**, same names, so the two documents line up.
- **Schemas with every field**, type, constraint and reference.
- **Every endpoint as a table row**: method, path, middleware, body, response, errors.
- **Real file and function names** from the repository.
- **Frontend state too**: slices, hooks, which component reads what.
- **Validations and edge cases** per endpoint, with the HTTP code returned.
- **Update it when the code changes**; a stale LLD is worse than none.

Part A - High-Level Design

2. Purpose and users

JobPortal lets **job seekers** discover and apply to jobs and lets **recruiters** post jobs, manage companies and review applicants. An AI layer scores how well a resume matches a job and detects company logos automatically.

Role	Can do
Job seeker	Browse / search / filter jobs, view job details, AI Smart Match (0-100 with feedback and missing skills), one-click apply, track applied jobs, profile with resume upload
Recruiter	Dashboard stats, post and manage jobs, manage companies (AI logo), per-job applicants, all applications with contact details, AI match score per applicant, accept / reject
Both	Register, login, logout; JWT session in an httpOnly cookie; role-based routes

3. Architecture

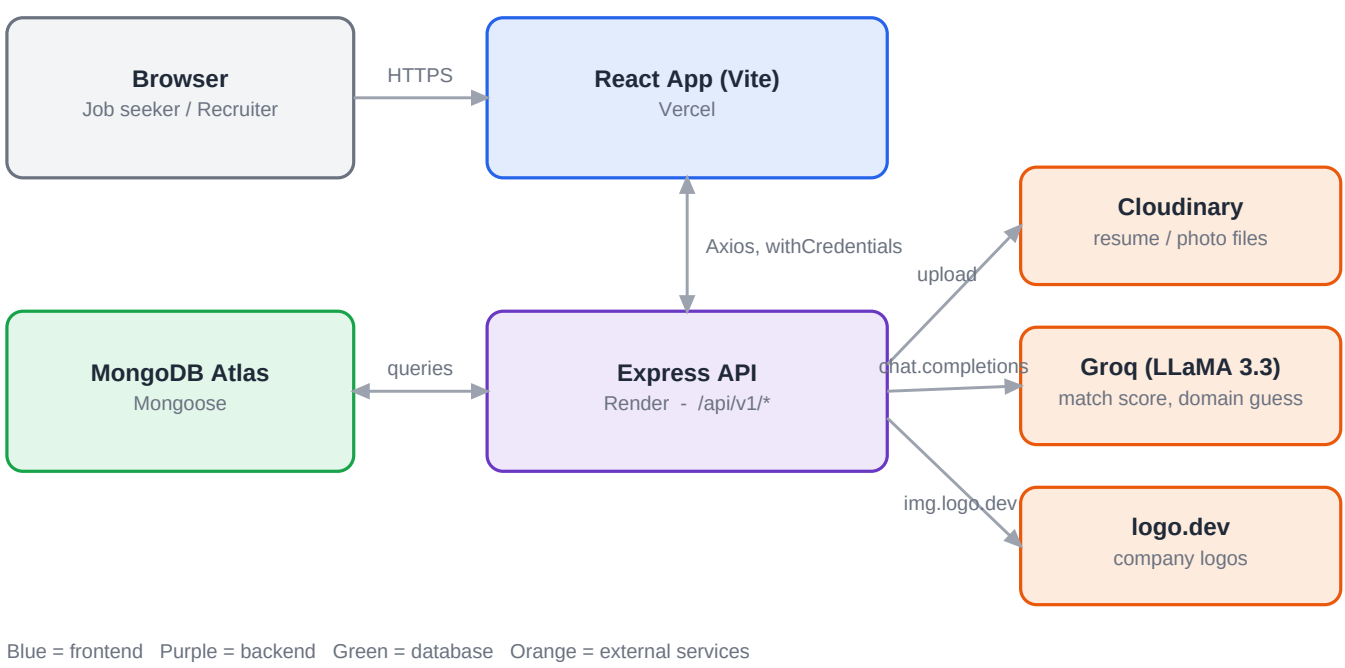


Figure 1 - System architecture. Client/server split with three external services.

4. Modules

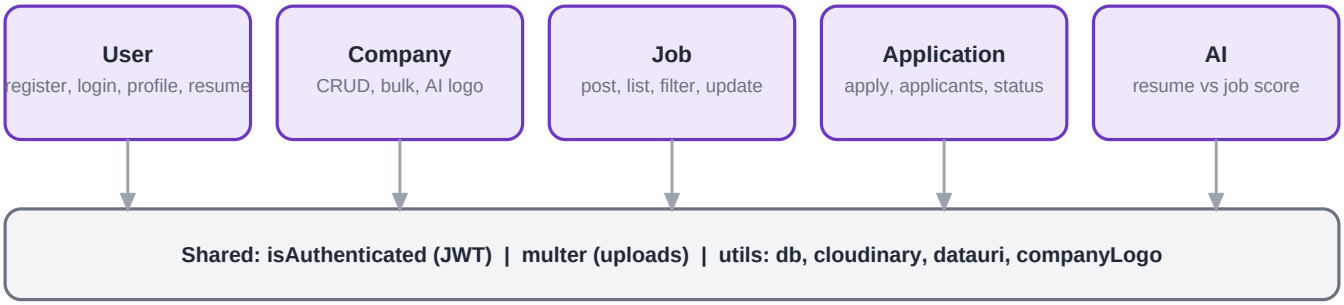


Figure 2 - Five backend modules on top of shared middleware and utilities. Each module = one route file + one controller file (+ a model where it owns data).

5. Key flows

5.1 Login

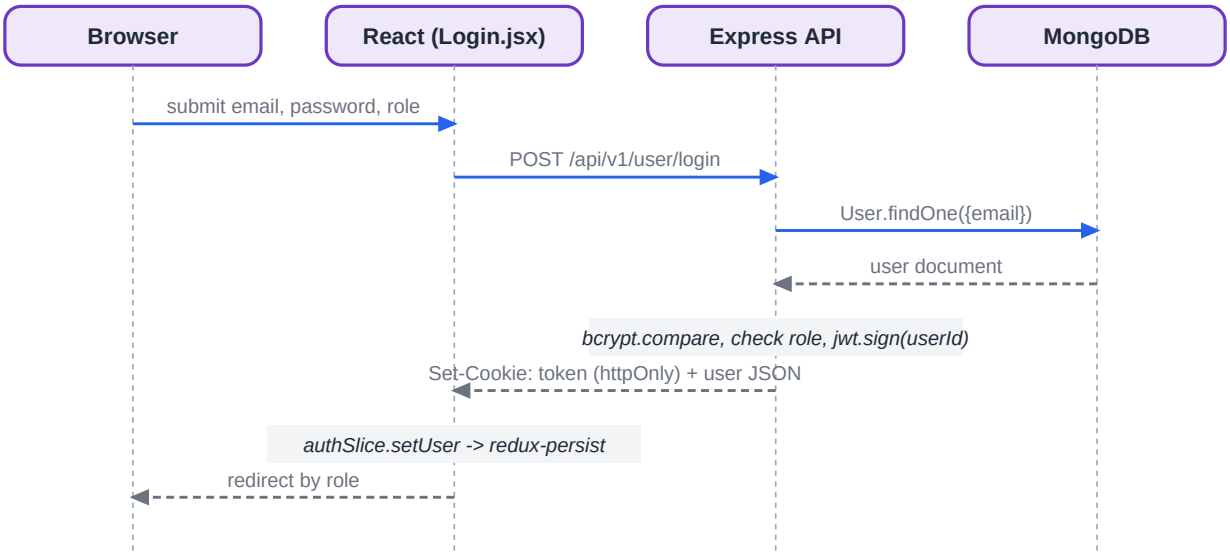


Figure 3 - Login. The token never reaches JavaScript; Axios sends it automatically with withCredentials.

5.2 Apply to a job

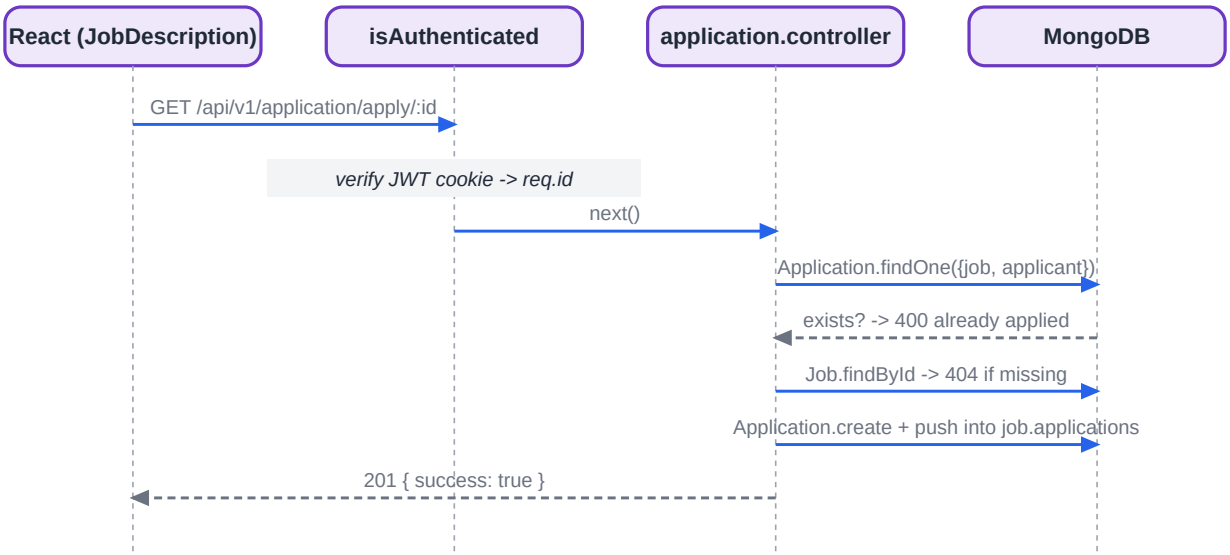


Figure 4 - Apply. Duplicate check happens before the job lookup.

5.3 AI Smart Match

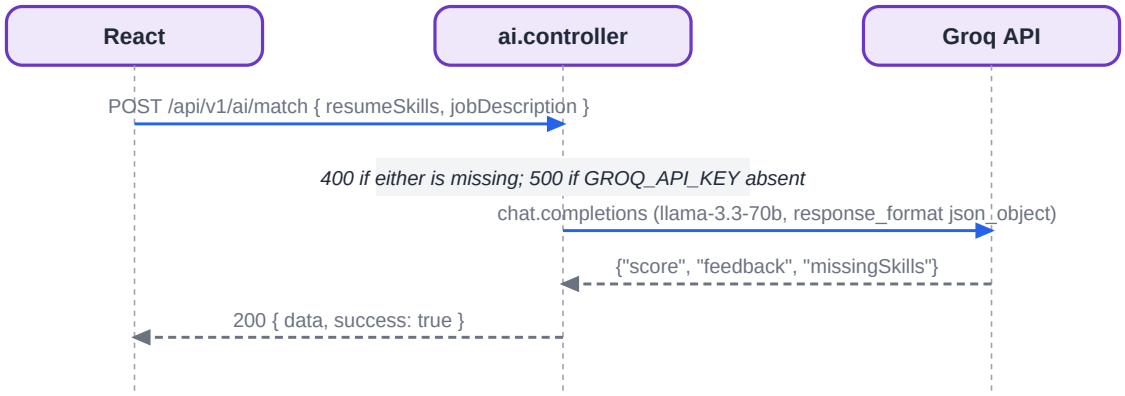


Figure 5 - Match scoring. The system prompt fixes the scoring rules; JSON mode guarantees a parseable reply.

5.4 Company logo detection (on company create / update)

- If the recruiter gave a website: extract the domain and use **img.logo.dev/<domain>**.
- If not: ask Groq to map the company name to its official domain, then use logo.dev.
- If neither works: fall back to a generated initials avatar (ui-avatars) so the image is never broken.

6. Data overview

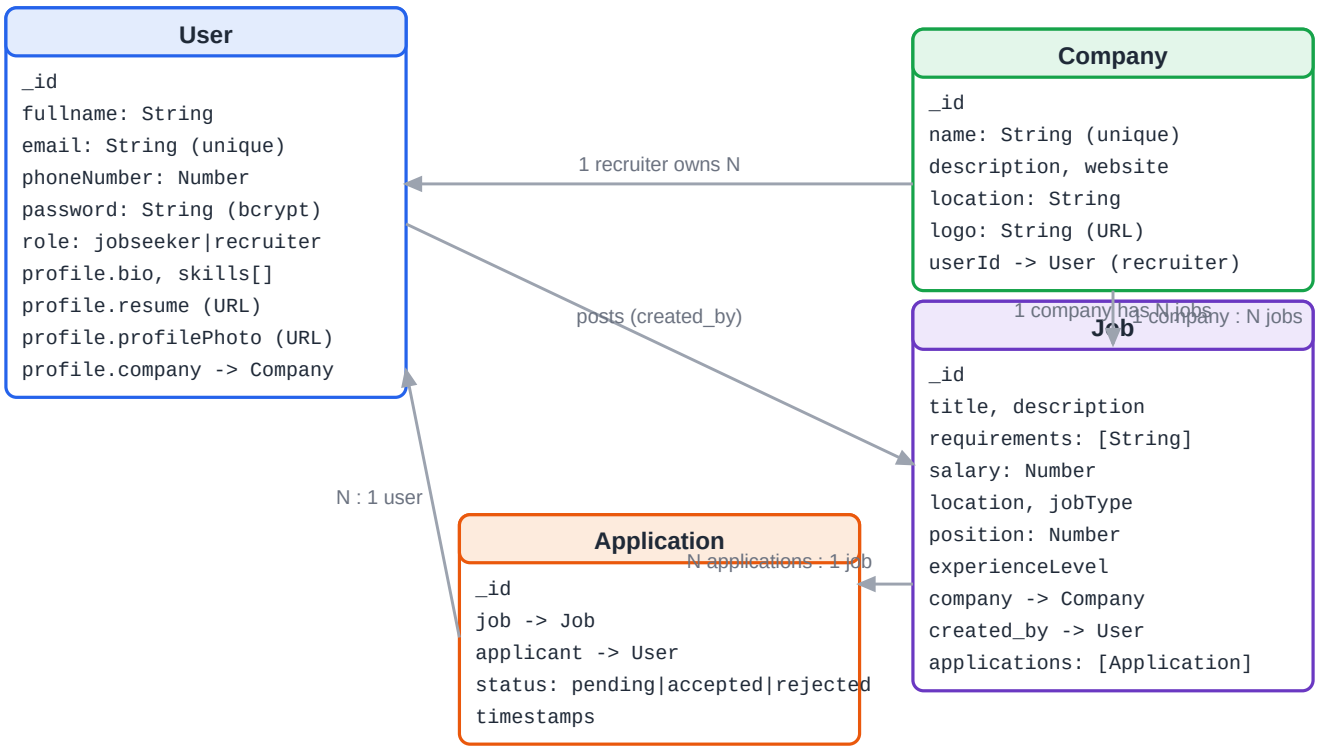


Figure 6 - Collections and references. Arrows point from the referencing field to the referenced collection.

7. Technology choices and reasons

Layer	Choice	Why
Frontend	React 19 + Vite, Redux Toolkit + redux-persist, React Router v7, Tailwind + Radix UI	Fast dev build; persisted auth survives refresh; accessible UI primitives

Layer	Choice	Why
Backend	Node + Express, Mongoose	Simple REST, one language across the stack, flexible document model
Auth	JWT signed with SECRET_KEY, stored in httpOnly cookie, bcrypt passwords	Cookie is not readable by JS (XSS-safe); stateless sessions
Files	Multer (memory) -> data URI -> Cloudinary	No disk on Render; CDN-hosted resumes and photos
AI	Groq, LLaMA 3.3 70B via OpenAI-compatible SDK	Fast and inexpensive; JSON mode for reliable parsing
Logos	logo.dev + Groq domain guess + initials fallback	Zero-upload company branding
Hosting	Vercel (frontend), Render (API), MongoDB Atlas	Free tiers, git-based deploys

8. Non-functional requirements

Area	Requirement / current behaviour
Security	Passwords hashed with bcrypt; JWT in httpOnly cookie; all write routes behind isAuthenticated; CORS with credentials
Roles	jobseeker / recruiter stored on User; frontend ProtectedRoute.jsx guards recruiter pages
Secrets	Only in backend .env / Render environment variables; never in the repo
Resilience	AI logo falls back to initials avatar; match endpoint returns 500 with a clear message if the key is missing
Uploads	Single file per request (multer singleUpload); stored on Cloudinary, URL kept on the user
Deployment	Vercel + Render + Atlas; backend health route GET / returns 'Backend is running...'

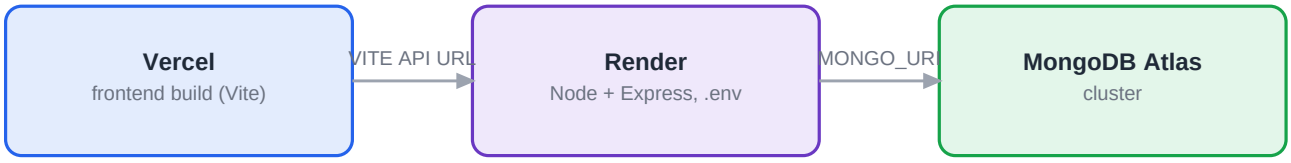
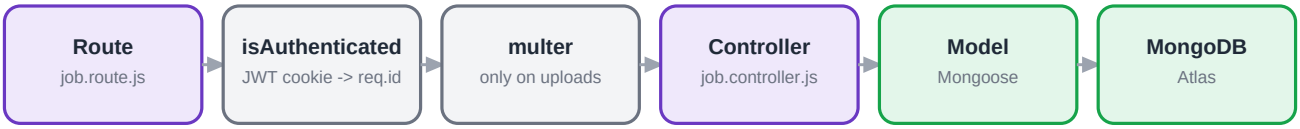


Figure 7 - Deployment.

Part B - Low-Level Design

9. Backend structure and request pipeline

```
backend/index.js
app.use(express.json, urlencoded, cookieParser, cors({origin:true, credentials:true}))
/api/v1/user -> routes/user.route.js
/api/v1/company -> routes/company.route.js
/api/v1/job -> routes/job.route.js
/api/v1/application -> routes/application.route.js
/api/v1/ai -> routes/ai.route.js
app.listen(PORT || 8000, connectDB)
```



Response: { success: Boolean, message: String, ...data } | 401 no cookie, 400 validation, 404 not found, 500 server

Figure 8 - Every request passes Route -> middleware -> controller -> model. Controllers always answer { success, message }.

Shared pieces

File	Responsibility	Key detail
middlewares/isAuthenticated.js	Reads req.cookies.token, verifies with SECRET_KEY, sets req.id = decoded.userId	401 'User not authenticated' when cookie missing; 401 'Invalid token' when verify fails
middlewares/multer.js	singleUpload: one file kept in memory	Used by register and profile/update
utils/db.js	connectDB(): mongoose.connect(MONGO_URI)	Logs connected DB name
utils/datauri.js	getDataUri(file) -> base64 data URI from multer buffer	Input for Cloudinary upload
utils/cloudinary.js	Configured Cloudinary v2 client	CLOUDINARY_CLOUD_NAME / API_KEY / API_SECRET
utils/companyLogo.js	extractDomain, guessDomainFromName (Groq), initialsAvatar	Returns a logo URL that is always valid

10. User module

Schema - models/user.model.js

Field	Type	Rules
fullname	String	required
email	String	required, unique
phoneNumber	Number	required
password	String	required, bcrypt hash
role	String enum	jobseeker | recruiter, required
profile.bio	String	optional
profile.skills	[String]	shown on profile, sent to AI match
profile.resume / resumeOriginalName	String	Cloudinary URL + original file name

Field	Type	Rules
profile.company	ObjectId -> Company	recruiter only
profile.profilePhoto	String	Cloudinary URL

Endpoints - routes/user.route.js -> controllers/user.controller.js

Method	Path	Middleware	Controller	Body / params	Errors
POST	/register	singleUpload	register	fullname, email, phoneNumber, password, role, file	400 missing fields / email exists
POST	/login	-	login	email, password, role	400 wrong credentials or role
GET	/logout	-	logout	-	clears cookie
POST	/profile/update	isAuthenticated, singleUpload	updateProfile	bio, skills, file (resume)	401; 400 user not found
DELETE	/profile/resume	isAuthenticated	deleteResume	-	401

Logic notes

- **login**: find by email -> bcrypt.compare -> role must match the stored role -> jwt.sign({userId}, SECRET_KEY, 1d) -> cookie *token* (httpOnly, sameSite strict) + sanitized user object.
- **updateProfile**: skills string is split on commas into an array; resume uploaded through getDataUri -> cloudinary.uploader.upload.

11. Company module

Schema - models/company.model.js

Field	Type	Rules
name	String	required, unique
description, website, location	String	optional
logo	String	URL; set by utils/companyLogo.js when not supplied
userId	ObjectId -> User	owning recruiter, required

Endpoints - routes/company.route.js -> controllers/company.controller.js

Method	Path	Middleware	Controller	Notes
POST	/register	isAuthenticated	registerCompany	400 if name missing or already exists; userId = req.id
GET	/get	isAuthenticated	getCompany	companies where userId = req.id
GET	/get/:id	isAuthenticated	getCompanyById	404 if not found
PUT	/update/:id	isAuthenticated, singleUpload	updateCompany	updates fields; resolves logo (logo.dev / Groq / avatar)
POST	/bulk	isAuthenticated	createMultipleCompanies	array of companies for the same recruiter

12. Job module

Schema - models/job.model.js

Field	Type	Rules
title, description	String	required
requirements	[String]	split from comma string on create
salary	Number	required; used by salary filter
location, jobType, experienceLevel	String	required; location and jobType used by filters

Field	Type	Rules
position	Number	open positions
company	ObjectId -> Company	required; populated in listings
created_by	ObjectId -> User	recruiter
applications	[ObjectId -> Application]	pushed on apply

Endpoints - routes/job.route.js -> controllers/job.controller.js

Method	Path	Middleware	Controller	Notes
POST	/post	isAuthenticated	postJob	400 if any field missing; created_by = req.id
GET	/get	-	getAllJobs	?keyword= regex on title/description; populate company; newest first; 404 if none
GET	/get/:id	-	getJobById	populate applications; 404 if missing
GET	/getadminjobs	isAuthenticated	getAdminJobs	jobs where created_by = req.id, populate company
PUT	/update/:id	isAuthenticated	updateJob	recruiter edits a posted job
POST	/bulk	isAuthenticated	postMultipleJobs	array insert for seeding

13. Application module

Schema - models/application.model.js

Field	Type	Rules
job	ObjectId -> Job	required
applicant	ObjectId -> User	required
status	String enum	pending (default) | accepted | rejected
createdAt / updatedAt	Date	timestamps: true

Endpoints - routes/application.route.js -> controllers/application.controller.js

Method	Path	Controller	Logic	Errors
GET	/apply/:id	applyJob	findOne({job, applicant}) -> create -> push id into job.applications	400 job id missing / already applied; 404 job
GET	/get	getAppliedJobs	find({applicant}) sorted desc, populate job -> company	404 none
GET	/:id/applicants	getApplicants	Job.findById populate applications -> applicant	404 job
GET	/recruiter/all	getAllApplicants	all applications across the recruiter's jobs, with email and phone	-
POST	/status/:id/update	updateStatus	body.status lower-cased -> accepted | rejected	400 status missing; 404 application

All five routes use isAuthenticated.

14. AI module

Method	Path	Controller	Input	Output
POST	/api/v1/ai/match	getMatchScore	{ resumeSkills: string, jobDescription: string }	{ data: { score 0-100, feedback, missingSkills[] }, success }

- Model **llama-3.3-70b-versatile** through the OpenAI SDK with baseURL https://api.groq.com/openai/v1.
- **response_format: json_object** so JSON.parse never fails on prose.
- System prompt: integer score, 100 = perfect skill overlap, do not penalise over-qualification.
- Validation order: 400 missing input -> 500 missing GROQ_API_KEY -> call -> 200.

15. Frontend design

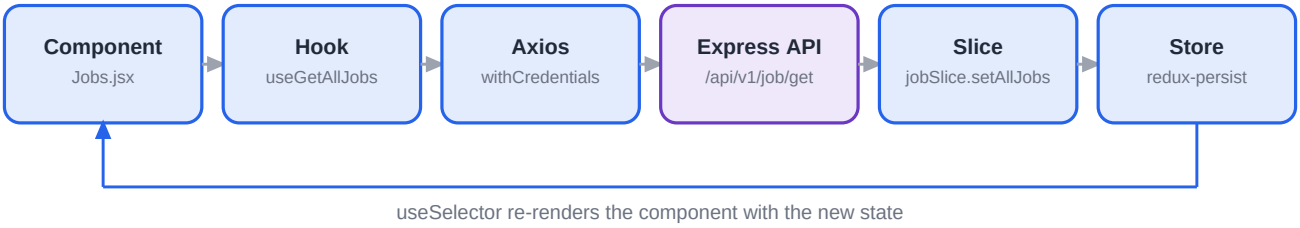


Figure 9 - Data flow for a list page. Hooks fetch once, slices hold the result, components only read the store.

Redux store - frontend/src/redux

Slice	State	Actions	Filled by
authSlice	loading, user	setLoading, setUser	Login.jsx / Signup.jsx / UpdateProfileDialog.jsx
jobSlice	allJobs, allAdminJobs, singleJob, allAppliedJobs, searchedQuery	setAllJobs, setAllAdminJobs, setSingleJob, setAllAppliedJobs, setSearchedQuery	useGetAllJobs, useGetAllAdminJobs, useGetAppliedJobs, JobDescription.jsx
companySlice	singleCompany, companies	setSingleCompany, setCompanies	useGetAllCompanies, useGetCompanyById
applicationSlice	applicants	setAllApplicants	Applicants.jsx / AllApplications.jsx

Pages and components

Area	Components	API used
Public / seeker	Home, HeroSection, CategoryCarousel, LatestJobs, LatestJobCards, Jobs, FilterCard, Job, Browse, JobDescription, AppliedJobs, AppliedJobTable, profile, UpdateProfileDialog	job/get, job/get/:id, application/apply/:id, application/get, user/profile/update, ai/match
Auth	auth/Login, auth/Signup	user/login, user/register
Recruiter (admin/)	ProtectedRoute, RecruiterProfile, Companies, CompaniesTable, CompanyCreate, CompanySetup, AdminJobs, AdminJobsTable, PostJob, Applicants, ApplicantsTable, AllApplications	company/*, job/getadminjobs, job/post, job/update/:id, application/:id/applicants, application/recruiter/all, application/status/:id/update, ai/match
Shared	shared/navbar, ui/* (Radix wrappers), Footer	user/logout

Conventions

- Base URLs live in **utils/constant.js** (USER_API_END_POINT, JOB_API_END_POINT, ...).
- Every Axios call sets **withCredentials: true** so the JWT cookie travels with it.
- **ProtectedRoute.jsx** redirects non-recruiters away from /admin/* pages.
- Filtering (location, industry, salary) is client-side on allJobs; keyword search goes to the API through searchedQuery.

16. Error handling and response contract

HTTP	Meaning in this project	Example
200 / 201	Success; body always has success: true	{ success: true, jobs: [...] }
400	Validation or business rule	'You have already applied for this job'
401	No / invalid JWT cookie	'User not authenticated'

HTTP	Meaning in this project	Example
404	Resource not found	'Job not found'
500	Unexpected error or missing configuration	'Server configuration error: AI Key missing.'

17. Known gaps (for the next iteration)

- isAuthenticated only logs on exception; it should return 401 inside catch so the request does not hang.
- apply is a GET that changes data; a POST would be the correct verb.
- No rate limiting or input size limits on the AI endpoint.
- Role checks are enforced in the UI, not on recruiter-only API routes.